Atlas — Fidelity Investments

Snowflake Advanced: Instructor Runbook & Hands-On Lab Guide

> **Audience:** Intermediate Snowflake users
> **Client:** Fidelity Investments — Equity & Options Trading Desk
> **Format:** Instructor-led, hands-on. 7 labs + capstone.
> **Environment:** Snowflake + a real AWS account (S3, SQS, IAM) + local Docker (Kafka).

0. Front Matter

0.1 The Atlas use case (read this to the class first)

Fidelity's Equity & Options Trading Desk is consolidating its market-data estate onto Snowflake. The platform — codenamed **Atlas** — must ingest market activity at **three different velocities** and serve it to analysts, risk, and compliance from one place:

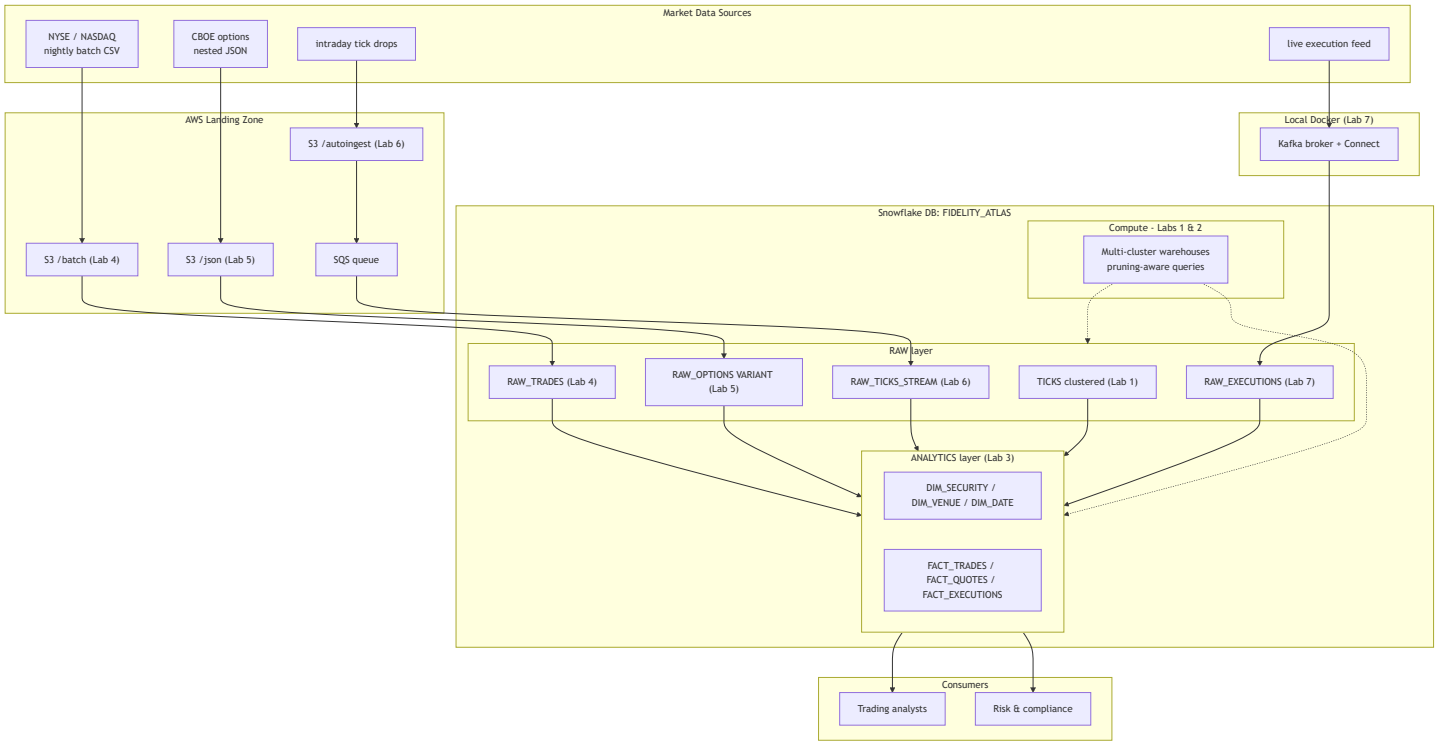
1. **Historical** — years of equity **ticks** for back-testing and TCA (transaction-cost analysis). Big, append-only, queried by date ranges.
2. **Batch** — nightly **trade files** dropped by venues (NYSE, NASDAQ, …) into S3.
3. **Micro-batch** — intraday tick files that land through the day and must be loaded **within seconds** (Snowpipe auto-ingest).
4. **Streaming** — a **live execution feed** (orders/fills) flowing through **Kafka**, landing in Snowflake with low latency (Snowpipe Streaming).

Along the way the data is messy **semi-structured JSON** (nested option chains) that must be modeled into clean star schemas.

Every lab builds one capability of Atlas, and **each lab consumes what the previous lab produced**. By the capstone, the class has built one coherent platform.

Lab	Capability	Headline skill	Built-in challenge
1	"Why are tick queries slow?"	Architecture & **micro-partition pruning**	Pruning experiments
2	"Market open overwhelms the desk"	**Warehouses**, scaling, concurrency	Class-wide concurrency test
3	"Analysts can't read raw feeds"	**Schemas & modeling**	The star-schema modeling challenge
4	"Onboard the NYSE batch venue"	**Bulk load + S3 integration**	Deliberate IAM trust break/fix
5	"Option chains are deeply nested"	**Semi-structured JSON**	The FLATTEN build challenge
6	"Nightly is too slow"	**Snowpipe auto-ingest (SQS)**	Dead-pipe debug
7	"The execution feed must stream"	**Kafka / Snowpipe Streaming**	Connector debug

0.2 Architecture (the whole platform on one page)



0.3 Prerequisites

- **Snowflake:** an account with `ACCOUNTADMIN` available to the instructor; each student gets a role with `CREATE` on the shared database, or their own database. Edition: **Enterprise or above** (multi-cluster warehouses in Lab 2).
- **AWS:** one account the class can use. Ability to create an **S3 bucket**, an **IAM role/policy**, and read **SQS** auto-created by Snowflake. (Labs 4, 6.)
- **Local:** Docker + Docker Compose, Python 3.10+, and `pip install kafka-python`.
- **Data:** generated by `generators/generate_market_data.py` (see 0.5).

0.4 Naming conventions (used in every lab)

```
Database     FIDELITY_ATLAS
Schemas      RAW         -- landing zone, 1:1 with sources
             ANALYTICS   -- modeled star schema for consumers
Warehouses   ATLAS_LOAD_WH      -- ingestion
             ATLAS_ANALYTICS_WH -- multi-cluster, query
Role         ATLAS_ENGINEER     -- students operate here
Stage prefix @ATLAS_...
```

> **Instructor note.** Set everything up under a dedicated database so teardown (capstone 8.3) is a single `DROP DATABASE`. Tell students to qualify objects as `RAW.TICKS`, not bare names, to avoid the classic "wrong schema" confusion.

0.5 Shared setup (run once, before Lab 1)

Generate the data locally:

```
cd generators
python generate_market_data.py          # ~14 MB across all files under ../data
# (use --quick for a fast smoke test with smaller volumes)
```

Bootstrap Snowflake objects all labs depend on:

```sql
-- Run as a role that can create databases (e.g. SYSADMIN/ACCOUNTADMIN).
CREATE DATABASE IF NOT EXISTS FIDELITY_ATLAS;
CREATE SCHEMA  IF NOT EXISTS FIDELITY_ATLAS.RAW;
CREATE SCHEMA  IF NOT EXISTS FIDELITY_ATLAS.ANALYTICS;

CREATE WAREHOUSE IF NOT EXISTS ATLAS_LOAD_WH
  WAREHOUSE_SIZE = 'XSMALL' AUTO_SUSPEND = 60 AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

USE DATABASE FIDELITY_ATLAS;
USE SCHEMA RAW;
USE WAREHOUSE ATLAS_LOAD_WH;
```

1. Lab 1 — Architecture & Micro-Partition Pruning

Business framing. *"Analysts complain that pulling one symbol for one week out of years of ticks takes far too long and burns credits. Why — and how do we fix it without indexes?"*

Time budget. Light exec/lecture; the time is in the **pruning experiments**.

1.1 Architecture refresher (5–10 min lecture)

Anchor three layers to the Atlas picture:

- **Cloud Services** — the "brain": SQL compilation, metadata, security, **and the pruning decisions** we are about to observe.
- **Compute (virtual warehouses)** — the muscle (Lab 2's whole topic).
- **Storage** — data sits in **micro-partitions**: contiguous, immutable units (~16 MB compressed, ~50–500 MB uncompressed conceptually). Snowflake keeps **per-partition metadata** (min/max of each column, distinct counts, nulls).

> **Key idea to land:** Snowflake has no indexes you create. Fast queries come from **pruning** — using per-partition min/max metadata to skip partitions that cannot contain matching rows. Our job as data engineers is to arrange data so the columns we filter on prune well.

1.2 Load the historical TICKS table

We load `ticks_history.csv` (**192,000 rows**, ~12 MB). It was written **sorted by SYMBOL** — deliberately *not* by date — so date-range filters prune poorly at

first. That sets up the experiment.

```
USE SCHEMA FIDELITY_ATLAS.RAW;

CREATE OR REPLACE TABLE TICKS (
  SYMBOL      STRING,
  TRADE_DATE  DATE,
  TICK_TS     TIMESTAMP_NTZ,
  LAST_PRICE  NUMBER(12,2),
  BID         NUMBER(12,2),
  ASK         NUMBER(12,2),
  TRADE_SIZE  NUMBER(10,0),
  VENUE       STRING
);

-- Stage the file (instructor uploads via Snowsight, or PUT from SnowSQL):
CREATE OR REPLACE STAGE ATLAS_LOCAL_STAGE
  FILE_FORMAT = (TYPE=CSV SKIP_HEADER=1 FIELD_OPTIONALLY_ENCLOSED_BY='"');

-- From SnowSQL/CLI:
--   PUT file://.../data/ticks_history.csv @ATLAS_LOCAL_STAGE;

COPY INTO TICKS
FROM @ATLAS_LOCAL_STAGE/ticks_history.csv
FILE_FORMAT = (TYPE=CSV SKIP_HEADER=1 FIELD_OPTIONALLY_ENCLOSED_BY='"')
ON_ERROR = 'ABORT_STATEMENT';

SELECT COUNT(*) AS rows_loaded, MIN(TRADE_DATE), MAX(TRADE_DATE) FROM TICKS;
-- expect 192000 rows, dates spanning ~3 months of weekdays
```

> **Snowsight upload alternative (no CLI).** Data → Databases → FIDELITY_ATLAS → RAW → Stages → ATLAS_LOCAL_STAGE → **+ Files**, upload the CSV, then run the `COPY INTO` above.

1.3 Experiment A — query *before* tuning, read the profile

```
-- A narrow slice: one symbol, one week.
ALTER WAREHOUSE ATLAS_LOAD_WH RESUME IF SUSPENDED;
USE WAREHOUSE ATLAS_LOAD_WH;

SELECT SYMBOL, AVG(LAST_PRICE) AS avg_px, SUM(TRADE_SIZE) AS vol
FROM TICKS
WHERE SYMBOL = 'NVDA'
  AND TRADE_DATE BETWEEN '2024-02-05' AND '2024-02-09'
GROUP BY SYMBOL;
```

Now inspect pruning **two ways**:

1. **Query Profile** (Snowsight → Query History → click the query → Profile tab). Look at the `TableScan` node: **Partitions scanned / Partitions total**.
2. **Programmatically**, capture the baseline:

```
SELECT
  query_id, partitions_scanned, partitions_total,
  ROUND(100*partitions_scanned/NULLIF(partitions_total,0),1) AS pct_scanned
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE query_text ILIKE '%NVDA%' AND query_text ILIKE '%2024-02-05%'
ORDER BY start_time DESC
LIMIT 5;
```

> **What students should see:** because rows are grouped by SYMBOL (not date), the `SYMBOL='NVDA'` predicate prunes well, but the **date predicate prunes poorly** — NVDA's ticks for that week are smeared across most of NVDA's partitions. Record the `partitions_scanned` number. **This is the baseline.**

1.4 Inspect natural clustering, then add a clustering key

```
-- How well is the table naturally clustered on the columns we filter?
SELECT SYSTEM$CLUSTERING_INFORMATION('TICKS', '(TRADE_DATE, SYMBOL)');
```

Read the JSON it returns with the class. The fields that matter:

- `average_overlaps` / `average_depth` — **higher = worse**; many partitions share the same `TRADE_DATE` ranges, so the engine can't skip them.
- `total_partition_count` — our universe of micro-partitions.

Now define a clustering key that matches the real query pattern (filter by date, then symbol):

```
ALTER TABLE TICKS CLUSTER BY (TRADE_DATE, SYMBOL);

-- Reclustering is asynchronous & automatic. To force the experiment to be
-- crisp in class, materialize a sorted copy so results are immediate:
CREATE OR REPLACE TABLE TICKS_CLUSTERED
  CLUSTER BY (TRADE_DATE, SYMBOL)
AS
SELECT * FROM TICKS ORDER BY TRADE_DATE, SYMBOL;
```

> **Instructor note.** Automatic reclustering can take time and credits; on a 192k-row table it may barely trigger. The `ORDER BY` rebuild above guarantees a well-clustered table *now* so the before/after contrast is visible within the class window. Explain that in production you'd rely on **automatic clustering** (a serverless feature) rather than rebuilding.

1.5 Experiment B — prove pruning improved

```
SELECT SYMBOL, AVG(LAST_PRICE) AS avg_px, SUM(TRADE_SIZE) AS vol
FROM TICKS_CLUSTERED
WHERE SYMBOL = 'NVDA'
  AND TRADE_DATE BETWEEN '2024-02-05' AND '2024-02-09'
GROUP BY SYMBOL;

-- Compare clustering quality before vs after:
SELECT SYSTEM$CLUSTERING_INFORMATION('TICKS_CLUSTERED', '(TRADE_DATE, SYMBOL)');
```

Re-open the Query Profile and compare **Partitions scanned / total** against the baseline from 1.3.

Metric	Before (TICKS)	After (TICKS_CLUSTERED)
Partitions scanned	high (most of NVDA's)	low (only the date-range's)
`average_depth`	larger	near 1
Bytes scanned	larger	smaller

> **Expected outcome:** the date-range query now touches a small fraction of partitions. That is pruning. **No index was created** — only the physical ordering changed.

1.6 Talking points & discussion

- Pruning is **predicate-driven**: it helps WHERE/JOIN keys, not SELECTed columns.
- Clustering keys cost money (reclustering compute). Choose them for **large, frequently-filtered** tables — exactly our `TICKS`. A 192k-row toy wouldn't warrant it in production; we use it to *see* the mechanic.
- Foreshadow Lab 2: "Now that one query is efficient, what happens when **80 analysts** run it at 9:30 a.m.?"

Deliverable from Lab 1 → used later: a `TICKS_CLUSTERED` table that Lab 3's model and the capstone query will read.

2. Lab 2 — Warehouses, Scaling & Concurrency

Business framing. *"At 9:30 a.m. the whole desk hits Atlas at once. Single queries are fast (Lab 1), but the dashboard still spins. Do we make the warehouse bigger, or do we make more of them?"*

Time budget. Scaling experiments + the **class-wide concurrency test**.

2.1 Warehouse fundamentals (lecture, 5 min)

Two independent dials — make sure the class never conflates them:

- **Size (scale UP):** XS → S → M → L … each step **doubles** credits/hour and roughly doubles compute for a *single* heavy query. Fixes **slow queries**.
- **Multi-cluster (scale OUT):** add *more clusters* of the same size that spin up under concurrency and spin down when idle. Fixes **queuing under load** — many users at once. (Enterprise edition.)

Symptom	Likely fix
One big aggregation is slow	Scale **up** (bigger size)
Queries **queue** when many users connect	Scale **out** (multi-cluster)
Both	Both, measured

2.2 Scaling experiment — UP

```
USE SCHEMA FIDELITY_ATLAS.RAW;

CREATE OR REPLACE WAREHOUSE ATLAS_ANALYTICS_WH
  WAREHOUSE_SIZE='XSMALL' AUTO_SUSPEND=60 AUTO_RESUME=TRUE INITIALLY_SUSPENDED=FALSE;
USE WAREHOUSE ATLAS_ANALYTICS_WH;

-- A deliberately heavy scan/aggregation over the historical ticks.
ALTER SESSION SET USE_CACHED_RESULT = FALSE;  -- so we measure real compute
SELECT SYMBOL, DATE_TRUNC('week', TRADE_DATE) AS wk,
       AVG(LAST_PRICE), MAX(ASK-BID) AS max_spread, SUM(TRADE_SIZE) AS vol
FROM TICKS_CLUSTERED
GROUP BY 1,2 ORDER BY 1,2;
-- Note the elapsed time from Query History.

ALTER WAREHOUSE ATLAS_ANALYTICS_WH SET WAREHOUSE_SIZE='MEDIUM';
-- Run the SAME query again; compare elapsed time.
```

> **Expected:** wall-clock drops materially going XS→M (more parallelism on the scan). Diminishing returns appear once the table fits comfortably — a great teaching moment about **not oversizing**. Reset to XS afterward.

2.3 Scaling experiment — OUT (multi-cluster)

```
ALTER WAREHOUSE ATLAS_ANALYTICS_WH SET
  WAREHOUSE_SIZE='XSMALL'
  MIN_CLUSTER_COUNT=1
  MAX_CLUSTER_COUNT=3
  SCALING_POLICY='STANDARD';
```

- `MIN=1, MAX=3` : idle cost = one XS cluster; under load Snowflake adds clusters.
- `SCALING_POLICY` : `STANDARD` favors responsiveness (spin up eagerly); `ECONOMY` favors credit savings (let queries queue a bit first).

2.4 The class-wide concurrency test

Goal: simulate market-open load and *watch the warehouse scale out live.*

Setup (instructor):

1. Grant the class `USAGE` on `ATLAS_ANALYTICS_WH` and `SELECT` on `TICKS_CLUSTERED` .
2. Hand every student the **same** query below.

3. Pick a **countdown** ("run on 3… 2… 1…") so everyone fires within a few seconds.

Student query (everyone runs simultaneously, several times):

```
USE WAREHOUSE ATLAS_ANALYTICS_WH;
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
SELECT SYMBOL, AVG(LAST_PRICE), SUM(TRADE_SIZE) AS vol,
       COUNT(*) AS ticks, MAX(ASK-BID) AS max_spread
FROM TICKS_CLUSTERED
WHERE TRADE_DATE BETWEEN '2024-01-15' AND '2024-03-15'
GROUP BY SYMBOL ORDER BY vol DESC;
```

Instructor observes (run repeatedly during the burst):

```
-- Live: how many clusters are running and is anything queuing?
SELECT * FROM TABLE(INFORMATION_SCHEMA.WAREHOUSE_LOAD_HISTORY(
  DATE_RANGE_START => DATEADD('minute',-15,CURRENT_TIMESTAMP()),
  WAREHOUSE_NAME   => 'ATLAS_ANALYTICS_WH'));
-- Watch AVG_RUNNING vs AVG_QUEUED_LOAD.

-- After the burst: per-query queue time.
SELECT query_id, user_name, execution_status,
       queued_provisioning_time, queued_overload_time, total_elapsed_time
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY_BY_WAREHOUSE('ATLAS_ANALYTICS_WH'))
WHERE start_time > DATEADD('minute',-15,CURRENT_TIMESTAMP())
ORDER BY start_time DESC;
```

> **The reveal.** With `MAX_CLUSTER_COUNT=1` (run a first round this way), students see `queued_overload_time` climb — queries waited. Bump to `MAX_CLUSTER_COUNT=3`, repeat the countdown, and watch additional clusters start (in `WAREHOUSE_LOAD_HISTORY`) while queue time collapses. **That contrast is the lesson.**

2.5 Reading the metrics & cost framing

- `AVG_QUEUED_LOAD > 0` → users are waiting → consider scaling **out**.
- Long `total_elapsed_time` with **no** queue → query is heavy → scale **up**.
- Multi-cluster bills **per active cluster-second**; auto-suspend keeps the idle bill at one cluster. Tie back to credits.

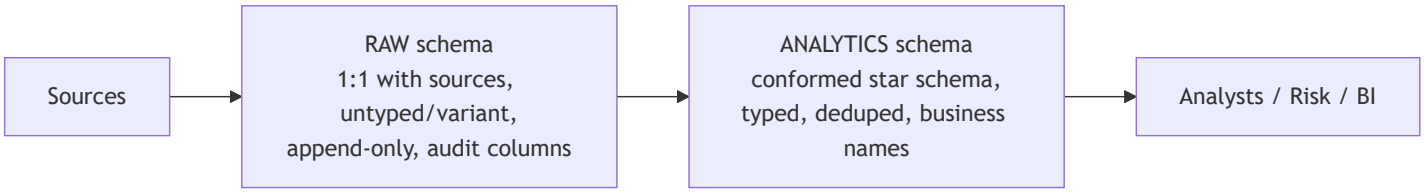
Deliverable from Lab 2 → used later: `ATLAS_ANALYTICS_WH` (multi-cluster) is the query warehouse for Lab 3 onward and the capstone.

3. Lab 3 — Schemas & Data Modeling

Business framing. *"Raw feeds are unusable for analysts: cryptic venue codes, no security master, ticks and trades in different shapes. Give us a clean, conformed model."*

Time budget. Quick DDL; the time is in the **modeling challenge**.

3.1 The layering strategy (lecture, 5 min)



- **RAW**: land data exactly as it arrives. Cheap to reload, full history, forgiving of source change. (TICKS, RAW_TRADES, RAW_OPTIONS, RAW_EXECUTIONS.)
- **ANALYTICS**: the **star schema** consumers actually query — conformed dimensions shared across every fact.

3.2 Quick DDL — conformed dimensions

```
USE SCHEMA FIDELITY_ATLAS.ANALYTICS;

CREATE OR REPLACE TABLE DIM_SECURITY (
  SECURITY_KEY  INT AUTOINCREMENT START 1 INCREMENT 1,
  SYMBOL        STRING NOT NULL,
  COMPANY_NAME  STRING,
  SECTOR        STRING,
  CONSTRAINT pk_dim_security PRIMARY KEY (SECURITY_KEY)
);

CREATE OR REPLACE TABLE DIM_VENUE (
  VENUE_KEY  INT AUTOINCREMENT START 1 INCREMENT 1,
  VENUE_CODE STRING NOT NULL,
  VENUE_NAME STRING,
  CONSTRAINT pk_dim_venue PRIMARY KEY (VENUE_KEY)
);

CREATE OR REPLACE TABLE DIM_DATE (
  DATE_KEY   INT,            -- yyyymmdd
  CAL_DATE   DATE,
  YEAR       INT, QUARTER INT, MONTH INT, DAY INT,
  DAY_OF_WEEK INT, IS_WEEKDAY BOOLEAN,
  CONSTRAINT pk_dim_date PRIMARY KEY (DATE_KEY)
);
```

> **Teaching note.** Snowflake constraints (except NOT NULL) are **not enforced** — they are metadata for tooling and the optimizer. Model them anyway: they document intent and drive BI tool joins.

Populate the dimensions from data we already have:

```
INSERT INTO DIM_SECURITY (SYMBOL, COMPANY_NAME, SECTOR)
SELECT DISTINCT t.SYMBOL,
       NULL, NULL          -- enriched below from a small security master
FROM FIDELITY_ATLAS.RAW.TICKS_CLUSTERED t;

-- Tiny security master (matches the generator's reference data):
MERGE INTO DIM_SECURITY d
USING (
  SELECT * FROM VALUES
    ('FNF','Fidelity National Financial','Financials'),
    ('AAPL','Apple Inc.','Technology'),
    ('MSFT','Microsoft Corp.','Technology'),
    ('JPM','JPMorgan Chase & Co.','Financials'),
    ('XOM','Exxon Mobil Corp.','Energy'),
    ('TSLA','Tesla Inc.','Consumer'),
    ('NVDA','NVIDIA Corp.','Technology'),
    ('BRKB','Berkshire Hathaway B','Financials')
  AS m(SYMBOL,COMPANY_NAME,SECTOR)
) s ON d.SYMBOL = s.SYMBOL
WHEN MATCHED THEN UPDATE SET d.COMPANY_NAME=s.COMPANY_NAME, d.SECTOR=s.SECTOR;

INSERT INTO DIM_VENUE (VENUE_CODE, VENUE_NAME)
SELECT * FROM VALUES
  ('NYSE','New York Stock Exchange'),('NASDAQ','NASDAQ'),
  ('CBOE','Cboe Options Exchange'),('ARCA','NYSE Arca'),('BATS','Cboe BZX');

INSERT INTO DIM_DATE
SELECT DISTINCT
  TO_NUMBER(TO_CHAR(d,'YYYYMMDD')) AS DATE_KEY, d AS CAL_DATE,
  YEAR(d), QUARTER(d), MONTH(d), DAY(d),
  DAYOFWEEK(d), DAYOFWEEK(d) BETWEEN 1 AND 5
FROM (
  SELECT DISTINCT TRADE_DATE AS d FROM FIDELITY_ATLAS.RAW.TICKS_CLUSTERED
);
```

3.3 The modeling challenge (students design this)

> **Prompt to the class:** *"Design the fact tables. We have quotes/ticks now, trades coming in Lab 4, options in Lab 5, executions in Lab 7. Decide the grain of each fact, which dimensions they share, and what measures they carry. You have 15 minutes — whiteboard it before you write DDL."*

Constraints to enforce during review:

- **Grain stated explicitly** for each fact (one row = ?).

- **Conformed dimensions** reused (no per-fact copies of security/venue/date).
- **Surrogate keys** in facts, not natural keys.
- Foreshadow that `FACT_QUOTES` (Lab 5) and `FACT_EXECUTIONS` (Lab 7) reuse the same dimensions.

3.4 A reference solution (reveal after the challenge)

```
-- Grain: one row per quote/tick observation.
CREATE OR REPLACE TABLE FACT_QUOTES (
  QUOTE_KEY    INT AUTOINCREMENT,
  SECURITY_KEY INT, VENUE_KEY INT, DATE_KEY INT,
  TICK_TS      TIMESTAMP_NTZ,
  LAST_PRICE   NUMBER(12,2), BID NUMBER(12,2), ASK NUMBER(12,2),
  SPREAD       NUMBER(12,4), TRADE_SIZE NUMBER(10,0)
);

-- Grain: one row per executed trade (Lab 4 fills this).
CREATE OR REPLACE TABLE FACT_TRADES (
  TRADE_KEY    INT AUTOINCREMENT,
  TRADE_ID     NUMBER,           -- natural key from venue file
  SECURITY_KEY INT, VENUE_KEY INT, DATE_KEY INT,
  EXEC_TS      TIMESTAMP_NTZ,
  SIDE         STRING, PRICE NUMBER(12,2), QUANTITY NUMBER(12,0),
  NOTIONAL     NUMBER(18,2)
);

-- Grain: one row per option-contract snapshot (Lab 5 fills this).
CREATE OR REPLACE TABLE FACT_OPTION_QUOTES (
  OPT_KEY      INT AUTOINCREMENT,
  SECURITY_KEY INT, DATE_KEY INT,
  CONTRACT_ID  STRING, OPT_TYPE STRING, STRIKE NUMBER(12,2),
  EXPIRATION   DATE, DAYS_TO_EXPIRY INT,
  LAST_PRICE   NUMBER(12,2), BID NUMBER(12,2), ASK NUMBER(12,2),
  VOLUME NUMBER, OPEN_INTEREST NUMBER,
  DELTA NUMBER(8,4), GAMMA NUMBER(8,4), THETA NUMBER(8,4),
  VEGA NUMBER(8,4), IMPLIED_VOL NUMBER(8,4)
);

-- Grain: one row per live execution event (Lab 7 fills this).
CREATE OR REPLACE TABLE FACT_EXECUTIONS (
  EXEC_KEY     INT AUTOINCREMENT,
  EVENT_ID     STRING,
  SECURITY_KEY INT, VENUE_KEY INT, DATE_KEY INT,
  EVENT_TS     TIMESTAMP_NTZ,
  SIDE STRING, ORDER_TYPE STRING, FILL_STATUS STRING,
  PRICE NUMBER(12,2), QUANTITY NUMBER(12,0), TRADER STRING
);
```

Now load `FACT_QUOTES` from the Lab 1 table to prove the model works end-to-end:

```
INSERT INTO FACT_QUOTES
  (SECURITY_KEY, VENUE_KEY, DATE_KEY, TICK_TS, LAST_PRICE, BID, ASK, SPREAD, TRADE_SIZE)
SELECT s.SECURITY_KEY, v.VENUE_KEY,
       TO_NUMBER(TO_CHAR(t.TRADE_DATE,'YYYYMMDD')),
       t.TICK_TS, t.LAST_PRICE, t.BID, t.ASK, (t.ASK - t.BID), t.TRADE_SIZE
FROM FIDELITY_ATLAS.RAW.TICKS_CLUSTERED t
JOIN DIM_SECURITY s ON s.SYMBOL = t.SYMBOL
JOIN DIM_VENUE    v ON v.VENUE_CODE = t.VENUE;

SELECT COUNT(*) FROM FACT_QUOTES;   -- 192000
```

Deliverable from Lab 3 → used everywhere after: the conformed dimensions and the four fact tables. Labs 4/5/7 simply populate the facts that are still empty.

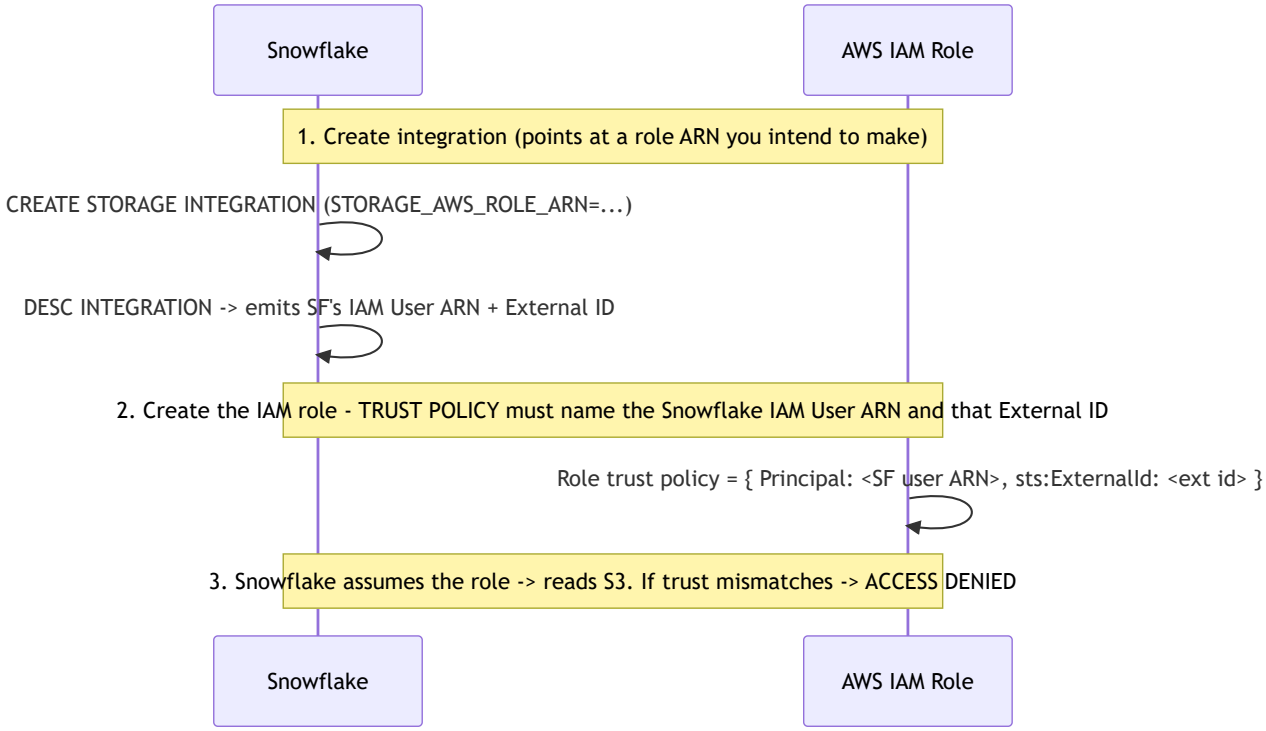
4. Lab 4 — Bulk Load + S3 Integration (with deliberate break/fix)

Business framing. *"Onboard the NYSE batch venue: nightly trade files land in our S3 bucket and must load into* `FACT_TRADES` *. First time we wire Snowflake to AWS — expect to get the trust relationship wrong."*

Time budget. Live AWS handshake + the **deliberate break/fix**.

4.1 The handshake explained (lecture, 5 min)

A **storage integration** is the secure, keyless bridge: Snowflake assumes an **IAM role in your AWS account**. The trust is mutual and circular, which is exactly why it's easy to misconfigure:



4.2 AWS side — bucket, policy, role

```
# (instructor/admin, AWS CLI) ------------------------------------------------
aws s3 mb s3://fidelity-atlas-batch-<unique>      # bucket
aws s3 cp data/trades_batch_2024-04-01.csv s3://fidelity-atlas-batch-<unique>/batch/
aws s3 cp data/trades_batch_2024-04-02.csv s3://fidelity-atlas-batch-<unique>/batch/
# (upload the rest similarly)
```

IAM **permissions policy** (least privilege — read + list the prefix):

```
{ "Version":"2012-10-17","Statement":[
  {"Effect":"Allow","Action":["s3:GetObject","s3:GetObjectVersion"],
   "Resource":"arn:aws:s3:::fidelity-atlas-batch-<unique>/batch/*"},
  {"Effect":"Allow","Action":["s3:ListBucket"],
   "Resource":"arn:aws:s3:::fidelity-atlas-batch-<unique>",
   "Condition":{"StringLike":{"s3:prefix":["batch/*"]}}}
]}
```

Create an IAM role `FidelityAtlasSnowflakeRole` with that permissions policy and a **placeholder trust policy** (we fix it in 4.4 — that's the planned break):

```
{ "Version":"2012-10-17","Statement":[
  {"Effect":"Allow","Principal":{"AWS":"arn:aws:iam::000000000000:root"},
   "Action":"sts:AssumeRole","Condition":{}}
]}
```

4.3 Snowflake side — integration, stage, COPY

```
USE SCHEMA FIDELITY_ATLAS.RAW;

-- Requires ACCOUNTADMIN (or a role granted CREATE INTEGRATION).
CREATE OR REPLACE STORAGE INTEGRATION ATLAS_S3_INT
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::<ACCOUNT_ID>:role/FidelityAtlasSnowflakeRole'
  STORAGE_ALLOWED_LOCATIONS = ('s3://fidelity-atlas-batch-<unique>/batch/');

-- This is the step students MUST do: read back the two values AWS needs.
DESC INTEGRATION ATLAS_S3_INT;
--   STORAGE_AWS_IAM_USER_ARN  -> arn:aws:iam::<snowflake-acct>:user/xxxx
--   STORAGE_AWS_EXTERNAL_ID   -> <external id string>

CREATE OR REPLACE FILE FORMAT FF_TRADES_CSV
  TYPE=CSV SKIP_HEADER=1 FIELD_OPTIONALLY_ENCLOSED_BY='"'
  TIMESTAMP_FORMAT='YYYY-MM-DD HH24:MI:SS';

CREATE OR REPLACE STAGE ATLAS_BATCH_STAGE
  STORAGE_INTEGRATION = ATLAS_S3_INT
  URL = 's3://fidelity-atlas-batch-<unique>/batch/'
  FILE_FORMAT = FF_TRADES_CSV;
```

4.4 The deliberate break/fix — IAM trust mismatch

Trigger the break. With the placeholder trust policy still in place, try to list the stage:

```
LIST @ATLAS_BATCH_STAGE;
```

Expected error:

```
Failure using stage area. Cause: [Access Denied (Status Code: 403; Error Code: AccessDenied)]
```

Diagnose with the class — the troubleshooting ladder:

1. *Is it auth or path?* 403/AccessDenied = **auth/trust**, not a bad URL (a bad path returns empty or NoSuchKey).

2. *Does the trust policy name the RIGHT principal?* Re-run `DESC INTEGRATION ATLAS_S3_INT;` and copy `STORAGE_AWS_IAM_USER_ARN` and `STORAGE_AWS_EXTERNAL_ID`.
   The placeholder `...:000000000000:root` with no external id cannot match.
3. *Did the External ID change?* It rotates if the integration is recreated — a classic real-world gotcha. Always re-`DESC` after any `CREATE OR REPLACE`.

The fix — correct the IAM role trust policy:

```
{ "Version":"2012-10-17","Statement":[
  {"Effect":"Allow",
   "Principal":{"AWS":"<STORAGE_AWS_IAM_USER_ARN from DESC>"},
   "Action":"sts:AssumeRole",
   "Condition":{"StringEquals":{"sts:ExternalId":"<STORAGE_AWS_EXTERNAL_ID from DESC>"}}}
]}
```

```
aws iam update-assume-role-policy \
  --role-name FidelityAtlasSnowflakeRole \
  --policy-document file://trust-policy-fixed.json
```

Verify the fix (allow ~30–60s for IAM propagation):

```
LIST @ATLAS_BATCH_STAGE;     -- now returns the trade files
```

> **Instructor note.** This is the single most common real Snowflake-on-AWS failure. Make students articulate *why* 403 means trust, not path. The rotating External ID is worth dwelling on — it bites people in production.

4.5 COPY INTO, validation, error handling

```
-- Dry run first: validate without loading.
COPY INTO FIDELITY_ATLAS.ANALYTICS.FACT_TRADES
FROM (
  SELECT
    NULL, $1, s.SECURITY_KEY, v.VENUE_KEY,
    TO_NUMBER(TO_CHAR(TO_TIMESTAMP_NTZ($2),'YYYYMMDD')),
    TO_TIMESTAMP_NTZ($2), $4, $5, $6, ($5 * $6)
  FROM @ATLAS_BATCH_STAGE t
  JOIN FIDELITY_ATLAS.ANALYTICS.DIM_SECURITY s ON s.SYMBOL = t.$3
  JOIN FIDELITY_ATLAS.ANALYTICS.DIM_VENUE    v ON v.VENUE_CODE = t.$7
)
FILE_FORMAT = FF_TRADES_CSV
VALIDATION_MODE = 'RETURN_ERRORS';   -- shows what WOULD fail; loads nothing
```

> **Note on column transforms.** When a `COPY` uses a `SELECT` (transform), the stage columns are positional (`$1..$7` matching the file's TRADE_ID,EXEC_TS,SYMBOL,SIDE,PRICE,QUANTITY,VENUE). The target column list maps in order; `TRADE_KEY` is AUTOINCREMENT so we pass `NULL` .

```
-- Real load.
COPY INTO FIDELITY_ATLAS.ANALYTICS.FACT_TRADES
  (TRADE_KEY, TRADE_ID, SECURITY_KEY, VENUE_KEY, DATE_KEY,
   EXEC_TS, SIDE, PRICE, QUANTITY, NOTIONAL)
FROM (
  SELECT NULL, t.$1, s.SECURITY_KEY, v.VENUE_KEY,
         TO_NUMBER(TO_CHAR(TO_TIMESTAMP_NTZ(t.$2),'YYYYMMDD')),
         TO_TIMESTAMP_NTZ(t.$2), t.$4, t.$5, t.$6, (t.$5 * t.$6)
  FROM @ATLAS_BATCH_STAGE t
  JOIN FIDELITY_ATLAS.ANALYTICS.DIM_SECURITY s ON s.SYMBOL = t.$3
  JOIN FIDELITY_ATLAS.ANALYTICS.DIM_VENUE    v ON v.VENUE_CODE = t.$7
)
FILE_FORMAT = FF_TRADES_CSV
ON_ERROR = 'CONTINUE';      -- skip bad rows, keep loading; review afterwards

-- What happened?
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME=>'FIDELITY_ATLAS.ANALYTICS.FACT_TRADES',
  START_TIME=>DATEADD('hour',-1,CURRENT_TIMESTAMP())));

SELECT COUNT(*) AS trades_loaded FROM FIDELITY_ATLAS.ANALYTICS.FACT_TRADES;
-- ~40,000 across the 5 daily files
```

- `ON_ERROR` options: `ABORT_STATEMENT` (default, all-or-nothing), `CONTINUE` , `SKIP_FILE` , `SKIP_FILE_<n>` .
- COPY tracks loaded files for ~64 days → re-running won't double-load (idempotency). Use `FORCE=TRUE` only to deliberately reload.

Deliverable from Lab 4 → used later: `FACT_TRADES` populated; the `ATLAS_S3_INT` storage integration is reused for the Lab 6 auto-ingest stage.

5. Lab 5 — Semi-Structured JSON (option chains)

Business framing. *"CBOE sends option chains as deeply nested JSON — underlying → expirations → calls/puts → contract → greeks. Land it raw, then shred it into `FACT_OPTION_QUOTES` without losing anything."*

Time budget. Experiments + a **build challenge**.

5.1 VARIANT & how Snowflake stores JSON (lecture, 5 min)

- `VARIANT` holds arbitrary JSON; Snowflake stores it **columnar-shredded** under the hood, so path access is fast and even prunes on sub-columns.
- Navigation operators: `:` for object keys, `[n]` for array elements, `::type` to cast. `FLATTEN` (a table function) explodes arrays into rows.

5.2 Load the nested payload

Our file `option_chains_2024-04-19.json` has **one JSON object per line** (5 underlyings, **240 contracts** total nested inside).

```
USE SCHEMA FIDELITY_ATLAS.RAW;

CREATE OR REPLACE FILE FORMAT FF_JSON TYPE=JSON STRIP_OUTER_ARRAY=FALSE;

CREATE OR REPLACE TABLE RAW_OPTIONS (
  V          VARIANT,
  LOADED_AT  TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Stage + load (Snowsight upload to a local stage, or PUT then COPY):
CREATE OR REPLACE STAGE ATLAS_JSON_STAGE FILE_FORMAT=FF_JSON;
--   PUT file://.../data/option_chains_2024-04-19.json @ATLAS_JSON_STAGE;

COPY INTO RAW_OPTIONS (V)
FROM @ATLAS_JSON_STAGE/option_chains_2024-04-19.json
FILE_FORMAT = FF_JSON;

SELECT COUNT(*) AS underlyings FROM RAW_OPTIONS;   -- 5
```

5.3 Navigation experiments (do these together)

```
-- Peek at the top level.
SELECT V:underlying:symbol::string  AS symbol,
       V:underlying:sector::string  AS sector,
       V:snapshotTs::timestamp_ntz  AS snapshot_ts,
       ARRAY_SIZE(V:expirations)     AS n_expirations
FROM RAW_OPTIONS;

-- One level down: explode the expirations array.
SELECT V:underlying:symbol::string AS symbol,
       e.value:expirationDate::date AS expiration,
       e.value:daysToExpiry::int    AS dte,
       ARRAY_SIZE(e.value:contracts:calls) AS n_calls,
       ARRAY_SIZE(e.value:contracts:puts)  AS n_puts
FROM RAW_OPTIONS,
     LATERAL FLATTEN(input => V:expirations) e;
```

> **Teaching beat:** each `FLATTEN` descends one array level. Two nested arrays (expirations → calls) need **two** FLATTENs, chained.

5.4 The build challenge (students write this)

> **Prompt:** *"Shred every individual contract — both calls and puts, across all expirations and underlyings — into `FACT_OPTION_QUOTES`, pulling the nested greeks up to columns. Join to `DIM_SECURITY` for the surrogate key. Expect 240 rows. Go."*

Hints to give only if stuck: "You need to FLATTEN expirations, then FLATTEN calls and puts separately and `UNION ALL` them (or FLATTEN a combined array)."

5.5 Reference solution

```
INSERT INTO FIDELITY_ATLAS.ANALYTICS.FACT_OPTION_QUOTES
 (SECURITY_KEY, DATE_KEY, CONTRACT_ID, OPT_TYPE, STRIKE, EXPIRATION,
  DAYS_TO_EXPIRY, LAST_PRICE, BID, ASK, VOLUME, OPEN_INTEREST,
  DELTA, GAMMA, THETA, VEGA, IMPLIED_VOL)
SELECT
  s.SECURITY_KEY,
  TO_NUMBER(TO_CHAR(r.V:snapshotTs::date,'YYYYMMDD')),
  c.value:contractId::string,
  c.value:type::string,
  c.value:strike::number(12,2),
  e.value:expirationDate::date,
  e.value:daysToExpiry::int,
  c.value:lastPrice::number(12,2),
  c.value:bid::number(12,2),
  c.value:ask::number(12,2),
  c.value:volume::number,
  c.value:openInterest::number,
  c.value:greeks:delta::number(8,4),
  c.value:greeks:gamma::number(8,4),
  c.value:greeks:theta::number(8,4),
  c.value:greeks:vega::number(8,4),
  c.value:greeks:impliedVol::number(8,4)
FROM RAW_OPTIONS r
JOIN FIDELITY_ATLAS.ANALYTICS.DIM_SECURITY s
     ON s.SYMBOL = r.V:underlying:symbol::string,
     LATERAL FLATTEN(input => r.V:expirations) e,
     LATERAL FLATTEN(input =>
        ARRAY_CAT(e.value:contracts:calls, e.value:contracts:puts)) c;

SELECT OPT_TYPE, COUNT(*) FROM FIDELITY_ATLAS.ANALYTICS.FACT_OPTION_QUOTES
GROUP BY OPT_TYPE;            -- 120 CALL + 120 PUT = 240
```

> `ARRAY_CAT` **trick:** concatenating the calls and puts arrays before a single FLATTEN is cleaner than two FLATTEN+UNION branches. Great thing to reveal after they've done it the verbose way.

5.6 Performance notes

- Filtering on a VARIANT path (`WHERE V:underlying:symbol::string='AAPL'`) can still prune, because Snowflake builds metadata on shredded sub-columns.
- For hot query paths, materialize into typed columns (exactly what `FACT_OPTION_QUOTES` is) — typed columns beat repeated path-casting at scale.

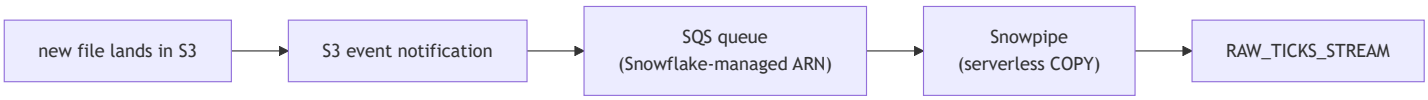
Deliverable from Lab 5 → used later: `FACT_OPTION_QUOTES` populated; the capstone joins it to the same dimensions as every other fact.

6. Lab 6 — Snowpipe Auto-Ingest (S3 → SQS, with dead-pipe debug)

Business framing. *"Nightly batch is too slow for the intraday desk. Tick files now drop into S3 through the day and must load within seconds — automatically, no human running COPY."*

Time budget. Live AWS (SQS) + **debugging a dead pipe**.

6.1 Snowpipe vs bulk load (lecture, 5 min)



- Bulk `COPY` = you run it, you size a warehouse, all-or-nothing batches.
- **Snowpipe** = event-driven, **serverless** (Snowflake-billed per-file compute), continuous, near-real-time. The S3 event → **SQS** → pipe chain is the whole trick, and the SQS wiring is where it breaks.

6.2 Create the pipe and get the SQS ARN

```
USE SCHEMA FIDELITY_ATLAS.RAW;

CREATE OR REPLACE TABLE RAW_TICKS_STREAM (
  SYMBOL STRING, TICK_TS TIMESTAMP_NTZ, LAST_PRICE NUMBER(12,2),
  TRADE_SIZE NUMBER(10,0), VENUE STRING,
  FILE_NAME STRING, LOADED_AT TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

CREATE OR REPLACE FILE FORMAT FF_TICKS_CSV
  TYPE=CSV SKIP_HEADER=1 FIELD_OPTIONALLY_ENCLOSED_BY='"'
  TIMESTAMP_FORMAT='YYYY-MM-DD HH24:MI:SS';

-- Reuse the Lab 4 storage integration; new prefix for the autoingest drops.
CREATE OR REPLACE STAGE ATLAS_AUTOINGEST_STAGE
  STORAGE_INTEGRATION = ATLAS_S3_INT
  URL = 's3://fidelity-atlas-batch-<unique>/autoingest/'
  FILE_FORMAT = FF_TICKS_CSV;

CREATE OR REPLACE PIPE ATLAS_TICKS_PIPE
  AUTO_INGEST = TRUE
AS
COPY INTO RAW_TICKS_STREAM (SYMBOL,TICK_TS,LAST_PRICE,TRADE_SIZE,VENUE,FILE_NAME)
FROM (
  SELECT $1,$2,$3,$4,$5, METADATA$FILENAME
  FROM @ATLAS_AUTOINGEST_STAGE
)
FILE_FORMAT = FF_TICKS_CSV;

-- THE critical value: Snowflake's SQS ARN for S3 to notify.
SHOW PIPES;
-- copy the notification_channel (arn:aws:sqs:...:.../sf-snowpipe-...)
```

> **Update the allowed locations** if needed so the integration permits the `/autoingest/` prefix:
>
> ```
> ALTER STORAGE INTEGRATION ATLAS_S3_INT SET STORAGE_ALLOWED_LOCATIONS =
>   ('s3://fidelity-atlas-batch-<unique>/batch/',
>    's3://fidelity-atlas-batch-<unique>/autoingest/');
> ```

6.3 Wire the S3 event notification to that SQS ARN

```
# S3 -> SQS event notification, scoped to the autoingest prefix + .csv suffix.
cat > s3-notify.json <<JSON
{ "QueueConfigurations":[{
    "Id":"atlas-snowpipe",
    "QueueArn":"<notification_channel ARN from SHOW PIPES>",
    "Events":["s3:ObjectCreated:*"],
    "Filter":{"Key":{"FilterRules":[
       {"Name":"prefix","Value":"autoingest/"},
       {"Name":"suffix","Value":".csv"}]}}
}]}
JSON

aws s3api put-bucket-notification-configuration \
  --bucket fidelity-atlas-batch-<unique> \
  --notification-configuration file://s3-notify.json
```

6.4 Trigger and watch auto-ingest

```
# Drop one intraday file; Snowpipe should pick it up within ~10-60s.
aws s3 cp data/ticks_intraday_2024-04-22_00.csv \
  s3://fidelity-atlas-batch-<unique>/autoingest/
```

```
-- Wait ~30-60s, then:
SELECT SYSTEM$PIPE_STATUS('ATLAS_TICKS_PIPE');
SELECT COUNT(*) FROM RAW_TICKS_STREAM;          -- ~2000 after first file
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME=>'FIDELITY_ATLAS.RAW.RAW_TICKS_STREAM',
  START_TIME=>DATEADD('hour',-1,CURRENT_TIMESTAMP())));
```

6.5 The dead-pipe debug (deliberate)

The break. Drop a second file but **misconfigure the notification first** — e.g. the prefix filter says `autoingest/` but you upload to `intraday/`, *or* the QueueArn was the wrong pipe's channel. Nothing loads. Students must diagnose a **silent** failure (no error is thrown anywhere obvious).

```
aws s3 cp data/ticks_intraday_2024-04-22_01.csv \
  s3://fidelity-atlas-batch-<unique>/intraday/   # wrong prefix on purpose
```

The diagnostic ladder:

```
-- 1. Is the pipe even running, and is it receiving messages?
SELECT SYSTEM$PIPE_STATUS('ATLAS_TICKS_PIPE');
--    Look at: executionState (should be RUNNING),
--             pendingFileCount, lastReceivedMessageTimestamp,
--             numOutstandingMessagesOnChannel.
--    lastReceivedMessageTimestamp NOT advancing => S3 events aren't arriving
--    => the problem is in AWS (notification), not Snowflake.

-- 2. Did any COPY happen for the new file?
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME=>'FIDELITY_ATLAS.RAW.RAW_TICKS_STREAM',
  START_TIME=>DATEADD('hour',-1,CURRENT_TIMESTAMP())));
--    No new row for the file => it was never notified or never matched the stage path.

-- 3. Cross-check the stage path vs where the file actually is.
LIST @ATLAS_AUTOINGEST_STAGE;   -- file in intraday/ won't appear under autoingest/
```

Root-cause logic to walk through:

- `executionState = RUNNING` **and** `lastReceivedMessageTimestamp` stale → Snowflake is healthy; **S3 is not notifying** → AWS-side issue.
- The file is under `intraday/` but the stage URL (and the S3 filter) target `autoingest/` → **path/prefix mismatch**.

The fix (pick whichever matches the break you used):

```
# Option A: put the file where the pipe is actually watching.
aws s3 cp data/ticks_intraday_2024-04-22_01.csv \
  s3://fidelity-atlas-batch-<unique>/autoingest/

# Option B: if the QueueArn was wrong, re-copy the notification_channel from
# SHOW PIPES and re-apply put-bucket-notification-configuration.


-- If files already sat in the stage before the channel was fixed, replay them:
ALTER PIPE ATLAS_TICKS_PIPE REFRESH;
SELECT COUNT(*) FROM RAW_TICKS_STREAM;     -- climbs as the rest of the 6 files load
```

> **Instructor note.** The teaching gold here is that Snowpipe failures are **silent**: no exception, just no data. `SYSTEM$PIPE_STATUS` is the single most important diagnostic — `lastReceivedMessageTimestamp` tells you instantly whether the failure is upstream (AWS) or in Snowflake. `ALTER PIPE … REFRESH` is the recovery valve for files that landed while the wiring was broken.

6.6 Cost & latency

- Snowpipe bills serverless credits **per file** plus a small overhead — so **many tiny files are expensive**. Coach students toward right-sized files (MBs, not KBs) in production.
- Typical latency: seconds to ~a minute. For sub-second, you need **streaming** — which is Lab 7.

Deliverable from Lab 6 → used later: `RAW_TICKS_STREAM` continuously fed; the capstone unions it with historical and streaming data.

7. Lab 7 — Kafka / Snowpipe Streaming (with connector debug)

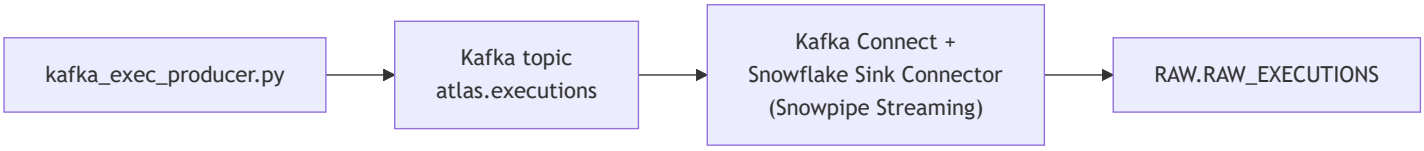
Business framing. *"The live execution feed — every order and fill — flows through Kafka. Risk needs it in Snowflake with **sub-second to seconds** latency. File-based Snowpipe isn't fast enough; use the Kafka connector in **Snowpipe Streaming** mode."*

Time budget. Most moving parts + **connector debugging**.

7.1 Streaming vs Snowpipe vs bulk (lecture, 5 min)

	Bulk COPY	Snowpipe (file)	Snowpipe **Streaming**
Trigger	manual	S3 event	rows pushed via SDK/connector
Unit	files	files	**rows**
Latency	minutes+	seconds	**sub-second–seconds**
Source here	Lab 4	Lab 6	**Kafka (Lab 7)**

The Kafka connector (Streaming mode) writes rows directly into the table channel — no intermediate S3 files.



7.2 Stand up Kafka in Docker + produce the feed

`docker-compose.yml` (single-broker KRaft, plus Kafka Connect):

```yaml
services:
  kafka:
    image: bitnami/kafka:3.7
    ports: ["9092:9092"]
    environment:
      - KAFKA_CFG_NODE_ID=1
      - KAFKA_CFG_PROCESS_ROLES=controller,broker
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
      - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
  connect:
    image: confluentinc/cp-kafka-connect:7.6.0
    depends_on: [kafka]
    ports: ["8083:8083"]
    environment:
      CONNECT_BOOTSTRAP_SERVERS: kafka:9092
      CONNECT_GROUP_ID: atlas-connect
      CONNECT_CONFIG_STORAGE_TOPIC: _connect_configs
      CONNECT_OFFSET_STORAGE_TOPIC: _connect_offsets
      CONNECT_STATUS_STORAGE_TOPIC: _connect_status
      CONNECT_CONFIG_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_OFFSET_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_STATUS_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_KEY_CONVERTER: org.apache.kafka.connect.storage.StringConverter
      CONNECT_VALUE_CONVERTER: org.apache.kafka.connect.json.JsonConverter
      CONNECT_VALUE_CONVERTER_SCHEMAS_ENABLE: "false"
      CONNECT_PLUGIN_PATH: /usr/share/java,/usr/share/confluent-hub-components
```

```
docker compose up -d
# Install the Snowflake connector plugin into the connect container:
docker exec -it $(docker ps -qf name=connect) \
  confluent-hub install --no-prompt snowflakeinc/snowflake-kafka-connector:latest
docker restart $(docker ps -qf name=connect)

# Create the topic and start producing (kafka-python):
pip install kafka-python
python generators/kafka_exec_producer.py \
  --bootstrap localhost:9092 --topic atlas.executions --rate 20 --count 2000
```

7.3 Snowflake side — key-pair auth + target table

Snowpipe Streaming requires **key-pair auth** (not password):

```
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub
```

```
USE SCHEMA FIDELITY_ATLAS.RAW;
CREATE OR REPLACE TABLE RAW_EXECUTIONS (
  RECORD_METADATA VARIANT,
  RECORD_CONTENT  VARIANT
);
-- Register the public key (paste the body of rsa_key.pub, no header/footer):
ALTER USER <streaming_user> SET RSA_PUBLIC_KEY='<contents of rsa_key.pub>';
GRANT USAGE ON DATABASE FIDELITY_ATLAS TO ROLE ATLAS_ENGINEER;
GRANT USAGE ON SCHEMA FIDELITY_ATLAS.RAW TO ROLE ATLAS_ENGINEER;
GRANT INSERT, SELECT ON TABLE RAW_EXECUTIONS TO ROLE ATLAS_ENGINEER;
```

7.4 Register the Snowflake sink connector (Streaming mode)

```
curl -X POST http://localhost:8083/connectors -H "Content-Type: application/json" -d '{
  "name": "atlas-exec-sink",
  "config": {
    "connector.class": "com.snowflake.kafka.connector.SnowflakeSinkConnector",
    "tasks.max": "1",
    "topics": "atlas.executions",
    "snowflake.url.name": "<account>.snowflakecomputing.com:443",
    "snowflake.user.name": "<streaming_user>",
    "snowflake.private.key": "<single-line contents of rsa_key.p8>",
    "snowflake.role.name": "ATLAS_ENGINEER",
    "snowflake.database.name": "FIDELITY_ATLAS",
    "snowflake.schema.name": "RAW",
    "snowflake.topic2table.map": "atlas.executions:RAW_EXECUTIONS",
    "snowflake.ingestion.method": "SNOWPIPE_STREAMING",
    "buffer.count.records": "100",
    "buffer.flush.time": "10",
    "buffer.size.bytes": "1000000",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false"
  }
}'
```

```
-- Within ~10-20s of producing, rows should appear:
SELECT COUNT(*) FROM RAW_EXECUTIONS;
SELECT RECORD_CONTENT:symbol::string AS sym,
       RECORD_CONTENT:side::string   AS side,
       RECORD_CONTENT:price::number(12,2) AS px,
       RECORD_CONTENT:fillStatus::string  AS status
FROM RAW_EXECUTIONS LIMIT 10;
```

7.5 The connector debug (deliberate)

Introduce **one** of these classic faults (instructor picks; all produce a connector that loads nothing):

1. **Auth:** paste the public key where the private key belongs, or include the `-----BEGIN-----` headers / line breaks in `snowflake.private.key` .
2. **Mapping:** `snowflake.topic2table.map` names a topic that doesn't match `topics` (typo `atlas.execution`).
3. **Converter/schema:** leave `value.converter.schemas.enable=true` while the producer sends schemaless JSON.

The diagnostic ladder:

```
# 1. Is the connector RUNNING or FAILED? Read the task trace.
curl -s http://localhost:8083/connectors/atlas-exec-sink/status | python -m json.tool
#    state=FAILED + a "trace" with the Java exception is your answer:
#      - JWT/keypair error            -> fault #1 (auth)
#      - table/channel not found      -> fault #2 (mapping)
#      - JsonConverter schema error   -> fault #3 (converter)

# 2. Tail the Connect worker logs for the stack trace.
docker logs $(docker ps -qf name=connect) --tail 100
```

```
-- 3. From Snowflake's side, is anything arriving at all?
SELECT COUNT(*) FROM RAW_EXECUTIONS;   -- 0 confirms nothing is landing
SHOW CHANNELS IN TABLE RAW_EXECUTIONS; -- streaming channels (empty if connector never connecte
```

The fix = correct the offending config, then redeploy the connector:

```
curl -X DELETE http://localhost:8083/connectors/atlas-exec-sink
# re-POST the corrected config from 7.4
curl -s http://localhost:8083/connectors/atlas-exec-sink/status | python -m json.tool  # RUNNI
```

> **Instructor note.** Mirror Lab 6's lesson: a streaming pipeline fails **silently in Snowflake** (just zero rows). The truth is in the **connector status endpoint and Connect logs**, not in Snowflake. Teach the reflex: *"no rows? check the connector status first, Snowflake second."* The private-key formatting fault (#1) is by far the most common in the field — the key must be a single line with no PEM header/footer.

7.6 Land executions in the model

```
-- Move streamed rows into the conformed FACT_EXECUTIONS (run on a schedule/Task in prod).
INSERT INTO FIDELITY_ATLAS.ANALYTICS.FACT_EXECUTIONS
 (EVENT_ID, SECURITY_KEY, VENUE_KEY, DATE_KEY, EVENT_TS,
  SIDE, ORDER_TYPE, FILL_STATUS, PRICE, QUANTITY, TRADER)
SELECT
  e.RECORD_CONTENT:eventId::string,
  s.SECURITY_KEY, v.VENUE_KEY,
  TO_NUMBER(TO_CHAR(e.RECORD_CONTENT:eventTs::timestamp_ntz,'YYYYMMDD')),
  e.RECORD_CONTENT:eventTs::timestamp_ntz,
  e.RECORD_CONTENT:side::string,
  e.RECORD_CONTENT:orderType::string,
  e.RECORD_CONTENT:fillStatus::string,
  e.RECORD_CONTENT:price::number(12,2),
  e.RECORD_CONTENT:quantity::number,
  e.RECORD_CONTENT:trader::string
FROM FIDELITY_ATLAS.RAW.RAW_EXECUTIONS e
JOIN FIDELITY_ATLAS.ANALYTICS.DIM_SECURITY s ON s.SYMBOL = e.RECORD_CONTENT:symbol::string
JOIN FIDELITY_ATLAS.ANALYTICS.DIM_VENUE    v ON v.VENUE_CODE = e.RECORD_CONTENT:venue::string;
```

Deliverable from Lab 7 → used in the capstone: `FACT_EXECUTIONS` fed from the live feed. All four ingestion velocities now land in one model.

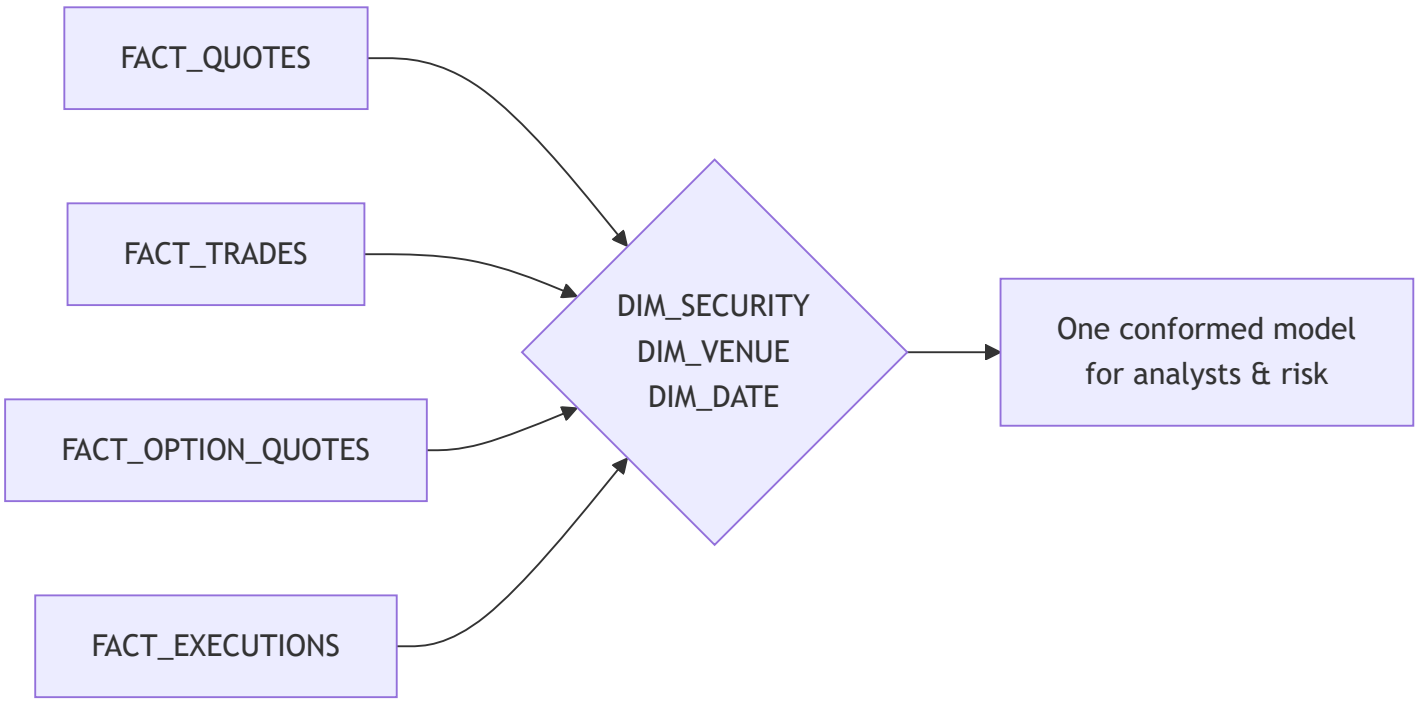
8. Capstone & Wrap-Up

8.1 The whole Atlas platform, running together

By now every ingestion path has populated the shared model:

Source velocity	Lab	Lands in	Modeled into
Historical ticks	1,3	`RAW.TICKS_CLUSTERED`	`FACT_QUOTES`

Source velocity	Lab	Lands in	Modeled into
Nightly trades (S3 bulk)	4	stage → COPY	`FACT_TRADES`
Option chains (JSON)	5	`RAW.RAW_OPTIONS`	`FACT_OPTION_QUOTES`
Intraday drops (Snowpipe/SQS)	6	`RAW.RAW_TICKS_STREAM`	(→ FACT_QUOTES in prod)
Live executions (Kafka stream)	7	`RAW.RAW_EXECUTIONS`	`FACT_EXECUTIONS`



8.2 Unified analyst query (the payoff)

A single query the trading desk could not run before Atlas — joining historical quotes, executed trades, and live executions through conformed dimensions:

```
USE WAREHOUSE ATLAS_ANALYTICS_WH;
USE SCHEMA FIDELITY_ATLAS.ANALYTICS;

WITH quote_activity AS (
  SELECT s.SYMBOL, COUNT(*) AS quote_ticks, AVG(q.SPREAD) AS avg_spread
  FROM FACT_QUOTES q JOIN DIM_SECURITY s USING (SECURITY_KEY)
  GROUP BY s.SYMBOL
),
trade_activity AS (
  SELECT s.SYMBOL, COUNT(*) AS trades, SUM(t.NOTIONAL) AS traded_notional
  FROM FACT_TRADES t JOIN DIM_SECURITY s USING (SECURITY_KEY)
  GROUP BY s.SYMBOL
),
exec_activity AS (
  SELECT s.SYMBOL, COUNT(*) AS live_execs,
         SUM(IFF(e.FILL_STATUS='FILLED',1,0)) AS filled
  FROM FACT_EXECUTIONS e JOIN DIM_SECURITY s USING (SECURITY_KEY)
  GROUP BY s.SYMBOL
),
options_activity AS (
  SELECT s.SYMBOL, COUNT(*) AS option_contracts, AVG(o.IMPLIED_VOL) AS avg_iv
  FROM FACT_OPTION_QUOTES o JOIN DIM_SECURITY s USING (SECURITY_KEY)
  GROUP BY s.SYMBOL
)
SELECT
  ds.SYMBOL, ds.COMPANY_NAME, ds.SECTOR,
  q.quote_ticks, ROUND(q.avg_spread,4) AS avg_spread,
  t.trades, t.traded_notional,
  x.live_execs, x.filled,
  o.option_contracts, ROUND(o.avg_iv,4) AS avg_implied_vol
FROM DIM_SECURITY ds
LEFT JOIN quote_activity   q USING (SYMBOL)
LEFT JOIN trade_activity   t USING (SYMBOL)
LEFT JOIN exec_activity    x USING (SYMBOL)
LEFT JOIN options_activity o USING (SYMBOL)
ORDER BY t.traded_notional DESC NULLS LAST;
```

> **Discussion:** point out that this one query spans data that arrived over months (historical), overnight (batch), through the day (micro-batch), and in the last few seconds (streaming) — all conformed to the same dimensions. That is the entire point of Atlas.

8.3 Teardown / cost cleanup (do this at the end)

```
-- Snowflake side: one drop removes everything.
DROP PIPE IF EXISTS FIDELITY_ATLAS.RAW.ATLAS_TICKS_PIPE;
DROP DATABASE IF EXISTS FIDELITY_ATLAS;
DROP WAREHOUSE IF EXISTS ATLAS_ANALYTICS_WH;
DROP WAREHOUSE IF EXISTS ATLAS_LOAD_WH;
-- Storage integration is account-level:
DROP INTEGRATION IF EXISTS ATLAS_S3_INT;


# AWS side:
aws s3 rb s3://fidelity-atlas-batch-<unique> --force
aws iam delete-role-policy --role-name FidelityAtlasSnowflakeRole --policy-name <name> 2>/dev/
aws iam delete-role --role-name FidelityAtlasSnowflakeRole
# Local:
docker compose down -v
```

> **Cost reminders to leave the class with:** suspend/auto-suspend warehouses; multi-cluster bills per active cluster; Snowpipe and Snowpipe Streaming bill serverless credits; many tiny files are the silent budget killer.

8.4 Instructor appendix — common student errors per lab

Lab	Symptom	Cause	Fix
1	Pruning "doesn't improve"	Result cache hid it; or auto-recluster hadn't run	`USE_CACHED_RESULT=FALSE` ; use the `ORDER BY` rebuild
1	`CLUSTERING_INFORMATION` errors	Column list quoting	Pass `'(TRADE_DATE, SYMBOL)'` exactly
2	No queuing observed	Class didn't fire together; cache on	Countdown; disable result cache; run multiple rounds
2	Multi-cluster won't scale	Standard (non-Enterprise) edition	Demonstrate scale-UP instead; explain edition limits
3	FACT load returns 0 rows	Dimension not populated first	Load DIM_SECURITY/VENUE before facts
4	`403 AccessDenied` on `LIST`	Trust policy principal/External ID wrong	Re- `DESC INTEGRATION` , fix trust policy (the lab)

Lab	Symptom	Cause	Fix
4	Works then breaks after recreate	External ID rotated on `CREATE OR REPLACE`	Re- `DESC` , update trust policy again
5	`FLATTEN` returns too few rows	Only flattened one array level	Chain FLATTENs; `ARRAY_CAT` calls+puts
5	Greeks are NULL	Wrong path casing (`impliedVol`)	Match JSON keys exactly (case-sensitive)
6	Pipe loads nothing, no error	S3→SQS notification wrong/prefix mismatch	`SYSTEM$PIPE_STATUS` ; fix notification; `ALTER PIPE … REFRESH`
6	Duplicate-looking loads	`REFRESH` after files already ingested	Snowpipe dedups by file; verify with `COPY_HISTORY`
7	Connector `FAILED`	Private key has PEM header/newlines	Single-line key, no header/footer
7	Connector RUNNING, 0 rows	Topic↔table map typo, or schemas.enable mismatch	Fix `topic2table.map` ; `schemas.enable=false`

8.5 What the class built

A single Snowflake platform — **Atlas** — that ingests market data at four velocities (historical, batch, micro-batch, streaming), lands messy JSON, models everything into one conformed star schema, and serves analysts and risk from a warehouse that scales with the desk. Each Snowflake Advanced topic was learned **in the place it actually matters** in a real trading-data pipeline.

Appendix A — File manifest

```
generators/
  generate_market_data.py     # all batch/historical/JSON/intraday data (seeded)
  kafka_exec_producer.py       # Lab 7 live execution feed -> Kafka
data/
  ticks_history.csv            # Lab 1   192,000 rows
  trades_batch_2024-04-0[1-5].csv   # Lab 4   ~8,000 rows each
  option_chains_2024-04-19.json     # Lab 5   5 underlyings / 240 contracts
  ticks_intraday_2024-04-22_0[0-5].csv  # Lab 6   2,000 rows each
```

Appendix B — Lab dependency chain (nothing is throwaway)